

Noi siamo interessati a contare il numero di macro-istruzioni necessarie.

Un modello comunque astratto, ma più espressivo con cui si può fare questa analisi è la **MACCHINA RAM**. A differenza dei modelli di calcolo precedenti, essa rientra nella famiglia delle **MACCHINE A REGISTRI**. I **REGISTRI** sono porzioni di memoria senza dimensione fissata*, a cui si accede con un **INDIRIZZO**, che contengono numeri naturali che non siano in base 1.

* occhio: Non sono array!

Sui registri si possono fare operazioni e gestire condizioni. Tutte le operazioni sono fatte nel primo registro, **RO**, detto **ACCUMULATORE**. Gli altri registri si usano per memorizzare i valori su cui operare.

Gli input e gli output delle **MACCH. RAM** sono sequenze di numeri naturali.

La differenza più grande tra le **M. RAM** e i modelli precedenti è che gli algoritmi delle **M. RAM** non sono definiti con palline e frecce, ma tramite **ISTRUZIONI...**

ESEMPIO:

READ 0	→	leggo l'input e lo metto nell'accumulatore
STORE 1	→	Metti il contenuto dell'accumulatore nel registro con indirizzo '1'
STORE 2	→	Analogo ma in R2
LOAD * 1	→	Carica in R0 il contenuto di R1
LOAD = 1	→	Carica (sempre in R0) il VALORE 1
STORE 3	→	Metti quel valore in R3

1 LOAD 1	→	Moltiplica il valore in R0 per quello in R3. Il risultato finisce sempre nell'accumulatore
MULT 3		
STORE 3		
LOAD 1		Sottrai il VALORE 1
SUB = 1		
STORE 2		Jump all'istruzione "etichettata" con "1" se il contenuto di R0 è Greater than zero
JGZ 1		
WRITE 3		Scrivi come output il contenuto di R3
HALT		

Termina l'esecuzione

Proviamo a valutarne la complessità.

- ① Quella temporale considera il numero di istruz. eseguite e il tempo per eseguire ciascuna istruzione.
Definiamo due tipi di complessità:

- $T_{\text{cost}}(n)$ in cui immagino che ogni istruzione impieghi lo stesso tempo per essere eseguita.

In questo caso:

- Eseguo le prime 5 istruzioni una volta
- // // ultime 2 // //
- le 7 in mezzo le eseguo tante volte quanto l'input

Dunque, $T_{\text{cost}}(n) = 7n + 7 = \Theta(n)$

- $T_{\text{log}}(n)$, dove si considera che il costo di ogni istruzione è proporzionale al numero di dati su cui opera (**CRITERIO DI COSTO LOGARITMICO**)
 Quindi $\text{READ } n \rightarrow l(n)$ dove l è il numero di bit per scrivere n in base 2. Invece,
 $\text{STORE } 2 \rightarrow l(n) + l(2)$ (considero anche l'indirizzo di 2 tra i dati su cui si opera), e
 $\text{LOAD } = 1 \rightarrow l(1)$
 scriviamole tutte ...

$\text{READ } 0 \quad l(n) + l(0)$

$\text{STORE } 1 \quad l(n) + l(1)$

$\text{STORE } 2 \quad l(n) + l(2)$

$\text{LOAD } = 1 \quad l(1)$

$\text{STORE } 3 \quad l(1) + l(3)$

1 $\text{LOAD } 1 \quad l(n) + l(1)$

$\text{MULT } 3 \quad l(n) + l(n^{i-1})$

$\text{STORE } 3 \quad l(n^i) + l(3)$

$\text{LOAD } 2 \quad l(n-i) + l(2)$

$\text{SUB} = 1 \quad l(n-i) + l(1)$

$\text{STORE } 2 \quad l(n-i-1) + l(2)$

$\text{JGZ } \mathbf{1} \quad l(n-i-1) + l(1)$

$\text{WRITE } 3 \quad l(n^n)$

$\text{HALT} \quad 1$

i indica la i -esima iterazione del ciclo

Nel calcolo del totale considero solo la parte dominante.

$$\begin{aligned}\text{Dunque ottengo } T_{\log}(n) &= O\left(\sum_{i=1}^n \ell(n^i)\right) = \\ &= O\left(\sum_{i=1}^n \log(n^i)\right) = O\left(\sum_{i=1}^n i \log(n)\right) = \\ &= O\left(\log(n) \sum_{i=1}^n i\right) = O\left(\log(n) \cdot \frac{n(n+1)}{2}\right) = \\ &= O\left(n^2 \log(n)\right)\end{aligned}$$

In generale, $T_{\log}(n) \geq T_{\text{cost}}(n)$, perché nel criterio di costo costante si fanno meno approssimazioni. Questo non significa che $T_{\log}$ sia realistico: anche qui abbiamo approssimato prendendo solo le istruzioni con maggiore tempo di esecuzione.

- ② Anche la complessità spaziale si misura in due modi diversi:
- Nel primo modo, prendo il numero di registri usati. In questo caso, otterremo $S_{\text{cost}}(n) = 4 = \Theta(1)$
 - Sappiamo che i registri possono avere dimensione qualsiasi. Una complessità più realistica considera le lunghezze dei vari registri, cioè la lunghezza del massimo valore rappresentato. Dunque $S_{\log}(n) = \Theta(n \log n)$

Allo fine, memorizziamo n^n , che si rappresenta con $\log_2 n^n = n \log_2 n \sim n \log n$ bit

Esistono anche dei simulatori delle macchine RAM online, per esempio
<https://random-access-machine-emulator.netlify.app>

①

Si consideri la seguente macchina RAM.

1	READ 1	7	STORE 2
2	LOAD= 1	8	LOAD 1
3	STORE 2	9	SUB= 1
4	loop: LOAD 1	10	STORE 1
5	JZ end	11	JUMP loop
6	MUL 2	12	end: WRITE 2
		13	HALT

- Si determini la funzione $f(n)$ calcolata.
- Si determini la complessità temporale della macchina in funzione dell'input n , $T_{RAM}(n)$, utilizzando un criterio di costo logaritmico.

SOLUZIONE:

Proviamo a simulare quello che fa la macchina RAM con input n ...

ISTRUZIONE	$M[0]$	$M[1]$	$M[2]$
1		n	
2	1	n	
3	1	n	1
4	n	n	1
5	(supponiamo $n \neq 0$ e vediamo cosa succede nel caso pessimo)		
6	n	n	1
7	n	n	n

8	n	n	n
9	$n-1$	n	n
10	$n-1$	$n-1$	n
11	(Torno indietro)		
12	} Non importanti per adesso		
13			

Quindi, il seguente algoritmo ripete il ciclo fino a quando non arrivo all'istruzione 5 con $M[1] = 0$

A ogni ciclo, sottraggo 1 al valore in $M[1]$ e lo moltiplico per il valore in $M[2]$. Quindi $M[2]$ sarà uguale a n , poi $n(n-1)$, poi $n(n-1)(n-2)$, e così via...

Una volta uscito dal ciclo, stamperei il valore in $M[2]$, che corrisponderebbe al fattoriale di n .

Vogliamo, adesso, calcolare la complessità temporale utilizzando i due criteri conosciuti

① CRITERIO DI COSTO COSTANTE

$$T(n) = 3 + n \cdot 8 + 2 + 2 = 8n + 7 = \Theta(n)$$

Istruzioni iniziali $\nearrow$ (pointing to 3)
 Ripeto il loop n volte $\nearrow$ (pointing to $n \cdot 8$)
 Istruzioni 4 e 5 prima di uscire $\nwarrow$ (pointing to 2)
 Istruzioni finali $\nwarrow$ (pointing to 2)

In questo caso, però, ci aspettiamo che il criterio di costo costante non ci dia una stima accurata, essendo $n!$ un numero molto grande $\rightarrow$ Alcune istruzioni costeranno di più

② CRITERIO DI COSTO LOGARITMICO

Usiamo la seguente approssimazione...

$$n! \sim \sqrt{2\pi n} \frac{n^n}{e^n}$$

Quindi, usando le proprietà dei logaritmi...

$$\log(n!) \sim \underbrace{n \log n}_{\Theta(n \log n)} - \underbrace{n \log e}_{\Theta(n)} + \underbrace{\frac{1}{2} \log n + \frac{1}{2} \log(2\pi)}_{\Theta(\log n)}$$

supponendo che n sia lungo $\log(n)$, valutiamo il costo di ciascuna istruzione in base all'operazione svolta

1	READ 1	$\log(n)$	7	STORE 2	$n \log(n)$
2	LOAD= 1	1	8	LOAD 1	$\log(n)$
3	STORE 2	1	9	SUB= 1	$\log(n)$
4	loop: LOAD 1	$\log(n)$	10	STORE 1	$\log(n)$
5	JZ end	1	11	JUMP loop	1
6	MUL 2	$\log(n) + \log(n!) =$ $= n \log^2(n)$	12	end: WRITE 2	$n \log(n)$
			13	HALT	1

Moltiplico qualcosa che è nell'ordine di grandezza di n per qualcosa nell'ordine di grandezza di $n!$

ordine di $n!$

Dunque, la complessità totale vale ...

$$T(n) = \log(n) + 2 + n \left[1 + n \log^2(n) + n \log(n) + 3 \log(n) + 1 \right] + n \log(n) + 3 = \Theta(n^2 \log^2(n))$$

Occhio a non dimenticare che il loop è ripetuto n volte!

2) Si realizzi una macchina RAM che, dati un array di interi A di lunghezza len e un intero n , ritorna 1 se n è presente nell'array, 0 altrimenti. Si supponga la memoria inizializzata nel seguente modo:

- L'array A è caricato in memoria
- $M[1]$ contiene l'indirizzo del primo elemento di A
- $M[2] = len$

Si determini la complessità temporale asintotica $T_{RAM}(n)$.

SOLUZIONE:

L'array non è dato ordinato (vedremo esercizi sugli ordinamenti più avanti nel corso) quindi l'unica idea è scorrere l'array da sinistra a destra cercando n ...

	READ	3	
loop:	LOAD *	1	← Carico l' i -esimo el. di A
	SUB	3	← Per verificare se è n ,
	JZ	found	gli sottraggo n e verifico
	LOAD	1	che il risultato sia 0
	ADD =	1	
	STORE	1	
	LOAD	2	
	SUB =	1	

Decremento
len

STORE 2

JZ notfound

JUMP loop

found: WRITE = 1

HALT

notfound: WRITE 0

HALT

se $len = 0$, allora ho
finito di scorrere A
senza trovare n

calcoliamo la complessità ...

- Se n non è presente nell'array ...

$$T(len) = 1 + len(11) + 2 = \Theta(len)$$

- Se n è presente nell'array ...

$$T(len) = 1 + k(11) + 3 + 2 \approx 1 + 11 \frac{len}{2} + 3 + 2 = \Theta(len)$$

In media, n si troverà
a metà

Quindi, in entrambi i casi, il costo è lo stesso
